

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB RECORD

Bio Inspired Systems (23CS5BSBIS)

Submitted by

Siddhant Sawhney (1BM23CS327)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING



B.M.S. COLLEGE OF ENGINEERING

(Autonomous Institution under VTU)

BENGALURU-560019

Aug-2025 to Jan-2026

B.M.S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering



CERTIFICATE

This is to certify that the Lab work entitled “ Bio Inspired Systems (23CS5BSBIS)” carried out by **Siddhant Sawhney (1BM23CS327)** who is bonafide student of **B.M.S. College of Engineering.**

It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum. The Lab report has been approved as it satisfies the academic requirements of the above mentioned subject and the work prescribed for the said degree.

Rohith Vaidya K, Mayanka Gupta (Batch Incharge) Assistant Professor Department of CSE, BMSCE	Dr. Kavitha Sooda Professor & HOD Department of CSE, BMSCE
--	--

Index

Sl. No.	Date	Experiment Title	Page No.
1	29/08/2025	Genetic Algorithm	1-8
2	29/08/2025	Gene Expression	9-14
3	12/09/2025	Particle Swarm Optimization	15-18
4	10/10/2025	Ant Colony Optimization	19-26
5	17/10/2025	Cuckoo Search	27-30
6	17/10/2025	Grey Wolf	31-34
7	7/11/2025	Parallel Cellular	35-38

Github Link:

https://github.com/siddhantsawhney327/327_BIS

Program 1

Genetic Algorithm

Algorithm:

Genetic Algorithm → 5 main phases

Steps:-

1.) Selecting Encoding Techniques

0 to 31

2.) Select the initial population - "4"

3.) S.No

	Initial Population	X value	Fitness $f(x) = x^2$	Prob $\frac{f(x)}{\sum f(x)}$	% Prob	Ex. Prob
1	0 1 1 0 0	12	144	0.1247	12.47	0.19
2	1 1 0 0 1	25	625	0.5411	54.11	2.104
3	0 0 1 0 1	5	25	0.216	2.16	0.036
4	1 0 0 1 1	19	361	0.3125	31.25	1.25

4.) Select Mating Pool

S.No	Mating Pool	Number of	Offspring	X	Fitness
1	0 1 1 0 0	1	0 1 1 0 1	13	169
2	1 1 0 0 1	1	1 1 0 0 1	24	576
3	1 1 0 0 1	2	1 1 0 1 1	27	729
4	1 0 0 1 1	1	1 0 0 0 1	17	289

5.) mutation

S.No.	offspring	mutation chromosome	offspring mutation	X	fitness $\frac{1}{1+ x }$ = $\frac{1}{1+2}$
1	01101	10000	11101	29	0.41
2	11000	00000	11000	24	0.76
3	11011	00000	11011	27	0.79
4	10001	00101	10100	20	0.80

Pseudocode

input encoding scheme
 generate a random population within the scheme
 find the fitness value
 find the expected count
 find actual value

Based on actual values select a mating pool

Find random crossover point for each pair
 in pool and perform crossover to generate
 offspring population

determine random mutation chromosome
 for each offspring and mutate
 find fitness

Repeat procedure till the max
 fitness of offspring is reached

code:

```
import random
```

```
pop = 6
```

```
BITS = 5
```

```
CROSS = 0.7
```

```
MUT = 0.1
```

```
SEED = 42
```

If SEED is not none:

```
random.seed(SEED)
```

```
def fit(s):
```

```
    return init(s, 2) * 2
```

```
def init - pop()
```

```
    return [''.join(random.choice('01') for  
        - in range(BITS)) for _ in  
        range(POP)]
```

```
def select(r):
```

```
    a, b = random.sample(pop, 2)
```

```
    return a if f(a) > f(b) else b
```

```
def cross(a, b):
```

```
    if random.random() < cross:
```

```
        pt = random.randint(1, BITS-1)
```

```
        return a[:pt] + b[pt:], b[:pt]
```

```
        return (a, b)
```

```
        + a[pt:]
```



```
def mutate(s):
```

```
    bits = len(s)
```

```
    for i in range(len(bits)):
```

```
        if random.random() < MUT:
```

```
            bits[i] = '1' if bits[i] == '0' else '0'
```

```
    return ''.join(bits)
```

```
def run(mom - stall=5):
```

```
    pop = init.pop()
```

```
    best, best_fit = None, -1
```

```
    stall = 0
```

```
    gen = 0
```

```
    while stall < mom - stall:
```

```
        gen += 1
```

```
        fit = [fit(n) for n in pop]
```

```
        gen_best = min(fit)
```

```
        gen_avg = sum(fit) /
```

```
        len(fit)
```

```
        best_idx = fit.index(gen_best)
```

```
        if gen_best > best_fit:
```

```
            best, best_fit = pop[best_idx],
```

```
            gen_best
```

```
            stall = 0
```

```
        else:
```

```
            stall += 1
```

```
    mut = []
```

```
    while len(mut) < pop:
```

```
        p1, p2 = select(pop)
```

```
        child = cross(
```

```
            p1, p2)
```

```

n = extend([mutate(c1), mutate(c2)])
pop = mut[:, pop]
print("\n Stopped after " + gen + " generation")
print("\n Genes best: " + best + " (n = " + mit(best, 2) + ") ,
      fit = " + bestfit)
run()

```

o/p

Gen 1 : Best = 01011	n = 11, fit = 121	Avg fit = 84.23
Gen 2 : Best = 01110	n = 14, fit = 196	Avg fit = 66.50
Gen 3 : Best = 01111	n = 14, fit = 224	Avg fit = 131.33
Gen 4 : Best = 01111	n = 15, fit = 225	Avg fit = 176.83
Gen 5 : Best = 01111	n = 14, fit = 224	Avg fit = 175.50
Gen 6 : Best = 11111	n = 31, fit = 961	Avg fit = 310.67
Gen 7 : Best = 11111	n = 31, fit = 961	Avg fit = 524.17
Gen 8 : Best = 11111	n = 31, fit = 961	Avg fit = 533.10
Gen 9 : Best = 11111	n = 31, fit = 961	Avg fit = 596.20
Gen 10 : Best = 11111	n = 31, fit = 961	Avg fit = 858.50
Gen 11 : Best = 11111	n = 31, fit = 961	Avg fit = 876.67

Stopped at 11 generation, Reporting Bestfit value of 961 for 5 consecutive generations

Final Best: 11111 (n = 31) fit = 961

Code:

```
import random

# Define the function to maximize
def fitness_function(x):
    return x ** 2

# Convert binary string to integer
def decode(chromosome):
    return int(chromosome, 2)

# Evaluate fitness for the entire population
def evaluate_population(population):
    return [fitness_function(decode(individual)) for individual in
population]

# Selection: Roulette Wheel
def select(population, fitnesses):
    total_fitness = sum(fitnesses)
    if total_fitness == 0:
        return random.choice(population)
    pick = random.uniform(0, total_fitness)
    current = 0
    for individual, fitness in zip(population, fitnesses):
        current += fitness
        if current > pick:
            return individual

# Crossover: Single-point crossover
def crossover(parent1, parent2):
    if random.random() < CROSSOVER_RATE:
        point = random.randint(1, CHROMOSOME_LENGTH - 1)
        return (parent1[:point] + parent2[point:], parent2[:point] +
parent1[point:])
    return parent1, parent2

# Mutation: Flip random bit
def mutate(chromosome):
    new_chromosome = ""
    for bit in chromosome:
        if random.random() < MUTATION_RATE:
            new_chromosome += '0' if bit == '1' else '1'
        else:
            new_chromosome += bit
    return new_chromosome

# Get user input for initial population
def get_initial_population(size, length):
    population = []
```

```

    print(f'Enter {size} chromosomes (each of {length} bits, e.g.,
    '10101'):")
    while len(population) < size:
        chrom = input(f'Chromosome {len(population)+1}: ").strip()
        if len(chrom) == length and all(bit in '01' for bit in chrom):
            population.append(chrom)
        else:
            print(f'Invalid input. Please enter a {length}-bit binary
            string.")
    return population

```

Run the Genetic Algorithm

```

def genetic_algorithm():
    population = get_initial_population(POPULATION_SIZE,
    CHROMOSOME_LENGTH)
    best_solution = None
    best_fitness = float('-inf')

```

```

    for generation in range(GENERATIONS):
        fitnesses = evaluate_population(population)

```

Update best solution

```

    for i, individual in enumerate(population):
        if fitnesses[i] > best_fitness:
            best_fitness = fitnesses[i]
            best_solution = individual

```

```

    print(f'Generation {generation + 1}: Best Fitness =
    {best_fitness}, Best x = {decode(best_solution)}")

```

```

    new_population = []
    while len(new_population) < POPULATION_SIZE:
        parent1 = select(population, fitnesses)
        parent2 = select(population, fitnesses)
        offspring1, offspring2 = crossover(parent1, parent2)
        offspring1 = mutate(offspring1)
        offspring2 = mutate(offspring2)
        new_population.extend([offspring1, offspring2])

```

```

    population = new_population[:POPULATION_SIZE]

```

```

    print("\nBest solution found:")
    print(f'Chromosome: {best_solution}')
    print(f'x = {decode(best_solution)}')
    print(f'f(x) = {fitness_function(decode(best_solution))}')

```

Parameters

POPULATION_SIZE = 4

CHROMOSOME_LENGTH = 5

```
MUTATION_RATE = 0.01
CROSSOVER_RATE = 0.8
GENERATIONS = 20
```

```
# Run it
if __name__ == "__main__":
    genetic_algorithm()
```

Enter 4 chromosomes (each of 5 bits, e.g., '10101'):

Chromosome 1: 01101

Chromosome 2: 11000

Chromosome 3: 11011

Chromosome 4: 10001

Generation 1: Best Fitness = 729, Best x = 27

Generation 2: Best Fitness = 729, Best x = 27

Generation 3: Best Fitness = 729, Best x = 27

Generation 4: Best Fitness = 729, Best x = 27

Generation 5: Best Fitness = 729, Best x = 27

Generation 6: Best Fitness = 729, Best x = 27

Generation 7: Best Fitness = 729, Best x = 27

Generation 8: Best Fitness = 729, Best x = 27

Generation 9: Best Fitness = 729, Best x = 27

Generation 10: Best Fitness = 729, Best x = 27

Generation 11: Best Fitness = 729, Best x = 27

Generation 12: Best Fitness = 729, Best x = 27

Generation 13: Best Fitness = 729, Best x = 27

Generation 14: Best Fitness = 729, Best x = 27

Generation 15: Best Fitness = 729, Best x = 27

Generation 16: Best Fitness = 729, Best x = 27

Generation 17: Best Fitness = 729, Best x = 27

Generation 18: Best Fitness = 729, Best x = 27

Generation 19: Best Fitness = 729, Best x = 27

Generation 20: Best Fitness = 729, Best x = 27

Best solution found:

Chromosome: 11011

x = 27

f(x) = 729

Program 2

Gene Expression

Algorithm:

lab 7

Genetic expression algorithm

Step 1

Fitness function: $f(n) = n^2$

Encoding Technique: 0 to 31

Use chromosome of fixed length (genotype)

Step 2

Partial Population

S.No	Initial Chromosome	Phenotype	Value	Fitness	P
1	+nn	n2	12	144	0.124
2	-nn	2n	25	625	0.541
3	n	n	5	25	0.216
4	-n2	n-2	19	361	0.302
Sum				1155	
avg				288.75	
mean				625	

Actual count expected val

1	0.5
2	2.1
3	0.08
4	1.25

Step 3: selection of mating pool

S.No	selected Chromosome	chromosome count	offspring	phenotype	n
1	+nn	2	nn	n2 (n+nn)	15
2	-nn	1	+nn	2n	25
3	+nn	3	+n-	n+ (n+nn)	27
4	-n2	1	-n2	n-2	17

Step 4: Crossover: Perform crossover randomly chosen gene
(not row 2, 15)

mean after crossover = 729

Step 5: Mutation

S-no	offspring before mutation	mutation applied	offspring after mutation	phenotype
1	n+	+ - -	n-	n+(n--)
2	+nn	none	+nn	2n
3	+n-	- - +	-n+	n+n+n
4	1n2	none	1n2	n+2

n value fitness

29 841

24 576

27 729

20 400

Step 6: Gene expression and evaluation

decode each genotype to phenotype
calculate fitness

$$\Sigma f(m) = 841 + 576 + 729 + 400 = 2546$$

$$\text{avg} = 636.5$$

$$\text{mean} = 841$$

Step 7: Iterate until convergence

Repeat step 3 to 6 fitness jump is
negligible

or generation limit has reached.

Pseudocode

Define fitness

Define parameters

Generate population

Select mating pool

Mutate

Gene expression and evaluation

Iterate till repeatedly value occurs

Output Best

Code:

```
import random
import math
```

```
# Example:  $f(x) = x * \sin(10 * \pi * x) + 2$ 
def fitness_function(x):
    return x * math.sin(10 * math.pi * x) + 2
```

```
POPULATION_SIZE = 6
GENE_LENGTH = 10
MUTATION_RATE = 0.05
CROSSOVER_RATE = 0.8
GENERATIONS = 20
DOMAIN = (-1, 2)
```

```
def random_gene():
    return random.uniform(DOMAIN[0], DOMAIN[1])
```

```
def create_chromosome():
    return [random_gene() for _ in range(GENE_LENGTH)]
```

```
def initialize_population(size):
    return [create_chromosome() for _ in range(size)]
```

```
def evaluate_population(population):
    return [fitness_function(express_gene(chrom)) for chrom in population]
```

```
def express_gene(chromosome):
    return sum(chromosome) / len(chromosome)
```

```
def select(population, fitnesses):
    total_fitness = sum(fitnesses)
    pick = random.uniform(0, total_fitness)
    current = 0
    for individual, fitness in zip(population, fitnesses):
        current += fitness
        if current > pick:
            return individual
    return random.choice(population)
```

```
def crossover(parent1, parent2):
    if random.random() < CROSSOVER_RATE:
        point = random.randint(1, GENE_LENGTH - 1)
        child1 = parent1[:point] + parent2[point:]
        child2 = parent2[:point] + parent1[point:]
        return child1, child2
    return parent1[:], parent2[:]
```

```
def mutate(chromosome):
    new_chromosome = []
    for gene in chromosome:
        if random.random() < MUTATION_RATE:
            new_chromosome.append(random_gene())
```

```

        else:
            new_chromosome.append(gene)
        return new_chromosome

def gene_expression_algorithm():
    population = initialize_population(POPULATION_SIZE)
    best_solution = None
    best_fitness = float("-inf")

    for generation in range(GENERATIONS):
        fitnesses = evaluate_population(population)

        for i, chrom in enumerate(population):
            if fitnesses[i] > best_fitness:
                best_fitness = fitnesses[i]
                best_solution = chrom[:]

        print(f"Generation {generation+1}: Best Fitness = {best_fitness:.4f}, Best x = {express_gene(best_solution):.4f}")

        new_population = []
        while len(new_population) < POPULATION_SIZE:
            parent1 = select(population, fitnesses)
            parent2 = select(population, fitnesses)
            offspring1, offspring2 = crossover(parent1, parent2)
            offspring1 = mutate(offspring1)
            offspring2 = mutate(offspring2)
            new_population.extend([offspring1, offspring2])

        population = new_population[:POPULATION_SIZE]

    print("\nBest solution found:")
    print(f"Genes: {best_solution}")
    x_value = express_gene(best_solution)
    print(f"x = {x_value:.4f}")
    print(f"f(x) = {fitness_function(x_value):.4f}")

if __name__ == "__main__":
    gene_expression_algorithm()

```

```

Generation 1: Best Fitness = 2.3125, Best x = 0.4262
Generation 2: Best Fitness = 2.3125, Best x = 0.4262
Generation 3: Best Fitness = 2.3125, Best x = 0.4262
Generation 4: Best Fitness = 2.3125, Best x = 0.4262
Generation 5: Best Fitness = 2.3125, Best x = 0.4262
Generation 6: Best Fitness = 2.3125, Best x = 0.4262
Generation 7: Best Fitness = 2.3125, Best x = 0.4262
Generation 8: Best Fitness = 2.4233, Best x = 0.6237
Generation 9: Best Fitness = 2.4233, Best x = 0.6237
Generation 10: Best Fitness = 2.4233, Best x = 0.6237
Generation 11: Best Fitness = 2.4233, Best x = 0.6237
Generation 12: Best Fitness = 2.4233, Best x = 0.6237

```

Generation 13: Best Fitness = 2.4233, Best x = 0.6237
Generation 14: Best Fitness = 2.4233, Best x = 0.6237
Generation 15: Best Fitness = 2.4233, Best x = 0.6237
Generation 16: Best Fitness = 2.4233, Best x = 0.6237
Generation 17: Best Fitness = 2.4395, Best x = 0.4594
Generation 18: Best Fitness = 2.4395, Best x = 0.4594
Generation 19: Best Fitness = 2.4395, Best x = 0.4594
Generation 20: Best Fitness = 2.4395, Best x = 0.4594

Best solution found:

Genes: [0.6948405045559576, -0.647173288232043, -0.3013499383055478, 1.631627548910124, 0.9271637073163099, 0.0324867196364278, -0.3565755055362756, 1.5226396608397925, 1.0654293190513275, 0.024805208657060707]

x = 0.4594

f(x) = 2.4395

Program 3

Particle Swarm Optimization

Algorithm:

Particle Swarm Optimisation (PSO)

pseudo code:

(1) p = particle initialized

(2) for $i=1$ to n :

for each particle p_i do:

$f = f(p)$

if f is better than $f(p_{best})$

$p_{best} \leftarrow p$

end if

end for

$g_{best} = \text{best } f_{ind}$

for each particle p_i do:

$$v_i^{t+1} = v_i^t + c_1 v_i^t (p_{best}^t - p_i^t) + c_2 (g_{best} - p_i^t)$$

$$p_i^{t+1} = p_i^t + v_i^{t+1}$$

end for

end for

as

Iteration 1

$$F(x, y) = x^2 + y^2$$

Inertia weight $(w) = 1$

Cognitive constant $(c_1) = 2$

Social constant $(c_2) = 2$

Random solution set to 1000

Iteration 1:

Particle	Position	Velocity	pbest	gbest	Fitness
P1	1, 1	0, 0	1, 1	1, 1	2
P2	-1, 1	0, 0	-1, 1		2
P3	0.5, -0.5	0, 0	0.5, -0.5		0.5
P4	-1, 1	0, 0	-1, 1		2
P5	0.25, 0.25	0, 0	0.25, 0.25		0.125

Best Fitness Value = 0.125 (P5)
 $so, g_{best} = 0.25, 0.25$

Iteration 2

Part no	Pos	Velocity	pbest	gbest	Fitness
P1	1, 1	-0.75, -0.25	1, 1	0.25, 0.25	2
P2	-1, 1	1.25, -0.25	-1, 1	0.25, 0.25	2
P3	0.5, -0.5	-0.5, -0.25	0.5, -0.5	0.25, 0.25	0.5
P4	-1, -1	-0.75, 0.25	-1, -1	0.25, 0.25	2
P5	0.25, 0.25	0, 0	0.25, 0.25	0.25, 0.25	0.125

~~gbest remains (0.25, 0.25)~~

Iteration 3

Particle no.	Pos	Velocity	pbest	gbest	Fitness
P1	1, 1	0.25, 0.25	1, 1	0.25, 0.25	2
P2	-1, 1	1.25, 0.25	-1, 1	0.25, 0.25	2
P3	0.5, -0.5	-0.5, 0.25	0.5, -0.5	0.25, 0.25	0.5
P4	-1, -1	-0.75, 0.25	-1, -1	0.25, 0.25	2
P5	0.25, 0.25	0, 0	0.25, 0.25	0.25, 0.25	0.125

Best position 0.25, 0.25
 Best fitness 0.125

Code:

```
import random
```

```
# Objective (fitness) function: De Jong function
```

```
def fitness_function(position):
```

```
    x, y = position
```

```
    return x*2 + y*2 # minimize this function
```

```
# PSO parameters
```

```
num_particles = 10
```

```
num_iterations = 50
```

```
W = 0.3 # inertia weight (from PDF)
```

```
C1 = 2 # cognitive coefficient
```

```
C2 = 2 # social coefficient
```

```
# Initialize particles and velocities
```

```
particles = [[random.uniform(-10, 10), random.uniform(-10, 10)] for _ in range(num_particles)]
```

```
velocities = [[0.0, 0.0] for _ in range(num_particles)]
```

```
# Initialize personal bests
```

```
pbest_positions = [p[:] for p in particles]
```

```
pbest_values = [fitness_function(p) for p in particles]
```

```
# Initialize global best
```

```
gbest_index = pbest_values.index(min(pbest_values))
```

```
gbest_position = pbest_positions[gbest_index][:]
```

```
gbest_value = pbest_values[gbest_index]
```

```
# PSO main loop
```

```
for iteration in range(num_iterations):
```

```
    for i in range(num_particles):
```

```
        r1, r2 = random.random(), random.random()
```

```
# Update velocity
```

```
velocities[i][0] = (W * velocities[i][0] +
```

```
C1 * r1 * (pbest_positions[i][0] - particles[i][0]) +
```

```
C2 * r2 * (gbest_position[0] - particles[i][0]))
```

```
velocities[i][1] = (W * velocities[i][1] +
```

```
C1 * r1 * (pbest_positions[i][1] - particles[i][1]) +
```

```
C2 * r2 * (gbest_position[1] - particles[i][1]))
```

```
# Update position
```

```
particles[i][0] += velocities[i][0]
```

```
particles[i][1] += velocities[i][1]
```

```
# Evaluate fitness
```

```
current_value = fitness_function(particles[i])
```

```
# Update personal best
```

```
if current_value < pbest_values[i]:  
    pbest_positions[i] = particles[i][:]  
    pbest_values[i] = current_value
```

```
# Update global best
```

```
if current_value < gbest_value:  
    gbest_value = current_value  
    gbest_position = particles[i][:]
```

```
print(f'Iteration {iteration+1}/{num_iterations} | Best Value: {gbest_value:.6f} at {gbest_position}')
```

```
print("\n☒ Optimal Solution Found:")  
print(f'Best Position: {gbest_position}')
```

```
print(f'Minimum Value: {gbest_value}')
```

```
Iteration 50/50 | Best Value: -44.591788 at [-10.113716514863546, -12.182177530114737]
```

```
☒ Optimal Solution Found:
```

```
Best Position: [-10.113716514863546, -12.182177530114737]
```

```
Minimum Value: -44.591788089956566
```

Ant Colony Optimization

LAB 3: First Colony Optimization

~~Initialize pheromone matrix with small positive values~~

Calculate semantic distance (inverse of distances)

Set parameters: num-ants, alpha, beta, rho, initial-
phenotype

- 4 initialize pointer handle blucity

- ↳ For each iteration:

4 for each and

- Start at a random city

4) Build a tour by choosing next city based on pheromone and distance

4. Evaluate all tours (calculate total dist)

↳ update phenomena:

4. Evaporate some phenol

47. Add pheromone based on four quality

Wish the best to our friend

2) of the shortest pair and distance

119.

o/p

Iteration 1 : Best length = 28.0355

Iteration 2 : Best length = 28.0355

.

.

.

Iteration 99 : Best length = 28.0355

Iteration 100 : Best length = 28.0355

Best path found:

[3, np.int64(5), np.int64(6), np.int64(11),
np.int64(0), np.int64(2), np.int64(4)]

Best path length: 28.0355

M9

10/10/25.

Code:

```
import numpy as np
```

```
class AntColony:
```

```
    def __init__(self, cities, n_ants, n_iterations, alpha=1.0, beta=5.0, rho=0.5,
pheromone_deposit=100):
```

```
        """
```

```
        cities: list of tuples (x, y) coordinates of cities
```

```
        n_ants: number of ants
```

```
        n_iterations: number of iterations to run
```

```
        alpha: pheromone importance
```

```
        beta: heuristic importance
```

```
        rho: evaporation rate
```

```
        pheromone_deposit: constant for pheromone deposit amount
```

```
        """
```

```
        self.cities = cities
```

```
        self.n = len(cities)
```

```
        self.n_ants = n_ants
```

```
        self.n_iterations = n_iterations
```

```
        self.alpha = alpha
```

```
        self.beta = beta
```

```
        self.rho = rho
```

```
        self.pheromone_deposit = pheromone_deposit
```

```
        # Calculate distance matrix
```

```
        self.dist_matrix = self.compute_distances()
```

```
        # Initialize pheromone trails to a small positive constant
```

```
        self.pheromone = np.ones((self.n, self.n))
```

```
        # Heuristic information (inverse of distance)
```

```
        self.eta = 1 / (self.dist_matrix + np.diag([np.inf]*self.n)) # avoid division by zero on diagonal
```

```
    def compute_distances(self):
```

```
        n = len(self.cities)
```

```
        dist = np.zeros((n, n))
```

```
        for i in range(n):
```

```
            for j in range(n):
```

```
                if i != j:
```

```
                    dist[i][j] = np.linalg.norm(np.array(self.cities[i]) - np.array(self.cities[j]))
```

```
        return dist
```

```
    def run(self):
```

```
        best_length = np.inf
```

```
        best_path = None
```

```
        for iteration in range(self.n_iterations):
```

```
            all_paths = []
```

```
            all_lengths = []
```

```
            for ant in range(self.n_ants):
```

```

        path = self.construct_solution()
        length = self.path_length(path)
        all_paths.append(path)
        all_lengths.append(length)

        if length < best_length:
            best_length = length
            best_path = path

        self.update_pheromones(all_paths, all_lengths)
        print(f'Iteration {iteration+1}: Best length = {best_length:.4f}')

    return best_path, best_length

def construct_solution(self):
    path = []
    visited = set()
    current_city = np.random.randint(0, self.n)
    path.append(current_city)
    visited.add(current_city)

    for _ in range(self.n - 1):
        next_city = self.select_next_city(current_city, visited)
        path.append(next_city)
        visited.add(next_city)
        current_city = next_city

    return path

def select_next_city(self, current_city, visited):
    pheromone = self.pheromone[current_city]
    heuristic = self.eta[current_city]

    prob_numerators = []
    for city in range(self.n):
        if city not in visited:
            prob = (pheromone[city] ** self.alpha) * (heuristic[city] ** self.beta)
        else:
            prob = 0
        prob_numerators.append(prob)

    prob_numerators = np.array(prob_numerators)
    total = prob_numerators.sum()
    if total == 0:
        # If all probabilities are zero (can happen), choose randomly among unvisited
        candidates = [c for c in range(self.n) if c not in visited]
        return np.random.choice(candidates)

    probabilities = prob_numerators / total

```



```

    next_city = np.random.choice(range(self.n), p=probabilities)
    return next_city

def path_length(self, path):
    length = 0
    for i in range(len(path) - 1):
        length += self.dist_matrix[path[i]][path[i+1]]
    # Return to the origin city
    length += self.dist_matrix[path[-1]][path[0]]
    return length

def update_pheromones(self, all_paths, all_lengths):
    # Evaporate pheromone
    self.pheromone *= (1 - self.rho)

    # Deposit new pheromone based on quality of solutions
    for path, length in zip(all_paths, all_lengths):
        deposit_amount = self.pheromone_deposit / length
        for i in range(len(path) - 1):
            self.pheromone[path[i]][path[i+1]] += deposit_amount
            self.pheromone[path[i+1]][path[i]] += deposit_amount # symmetric
        # add pheromone for the edge returning to the start city
        self.pheromone[path[-1]][path[0]] += deposit_amount
        self.pheromone[path[0]][path[-1]] += deposit_amount

if __name__ == "__main__":
    # Example set of cities (coordinates)
    cities = [
        (0, 0),
        (1, 5),
        (5, 2),
        (6, 6),
        (8, 3),
        (7, 9),
        (2, 7)
    ]

    ant_colony = AntColony(
        cities=cities,
        n_ants=10,
        n_iterations=100,
        alpha=1.0,
        beta=5.0,
        rho=0.5,
        pheromone_deposit=100
    )

    best_path, best_length = ant_colony.run()
    print("\nBest path found:")

```

```
print(best_path)
print(f'Best path length: {best_length:.4f}')
```

Iteration 1: Best length = 28.0355
Iteration 2: Best length = 28.0355
Iteration 3: Best length = 28.0355
Iteration 4: Best length = 28.0355
Iteration 5: Best length = 28.0355
Iteration 6: Best length = 28.0355
Iteration 7: Best length = 28.0355
Iteration 8: Best length = 28.0355
Iteration 9: Best length = 28.0355
Iteration 10: Best length = 28.0355
Iteration 11: Best length = 28.0355
Iteration 12: Best length = 28.0355
Iteration 13: Best length = 28.0355
Iteration 14: Best length = 28.0355
Iteration 15: Best length = 28.0355
Iteration 16: Best length = 28.0355
Iteration 17: Best length = 28.0355
Iteration 18: Best length = 28.0355
Iteration 19: Best length = 28.0355
Iteration 20: Best length = 28.0355
Iteration 21: Best length = 28.0355
Iteration 22: Best length = 28.0355
Iteration 23: Best length = 28.0355
Iteration 24: Best length = 28.0355
Iteration 25: Best length = 28.0355
Iteration 26: Best length = 28.0355
Iteration 27: Best length = 28.0355
Iteration 28: Best length = 28.0355
Iteration 29: Best length = 28.0355
Iteration 30: Best length = 28.0355
Iteration 31: Best length = 28.0355
Iteration 32: Best length = 28.0355
Iteration 33: Best length = 28.0355
Iteration 34: Best length = 28.0355
Iteration 35: Best length = 28.0355
Iteration 36: Best length = 28.0355
Iteration 37: Best length = 28.0355
Iteration 38: Best length = 28.0355
Iteration 39: Best length = 28.0355
Iteration 40: Best length = 28.0355
Iteration 41: Best length = 28.0355
Iteration 42: Best length = 28.0355
Iteration 43: Best length = 28.0355
Iteration 44: Best length = 28.0355
Iteration 45: Best length = 28.0355

Iteration 46: Best length = 28.0355
Iteration 47: Best length = 28.0355
Iteration 48: Best length = 28.0355
Iteration 49: Best length = 28.0355
Iteration 50: Best length = 28.0355
Iteration 51: Best length = 28.0355
Iteration 52: Best length = 28.0355
Iteration 53: Best length = 28.0355
Iteration 54: Best length = 28.0355
Iteration 55: Best length = 28.0355
Iteration 56: Best length = 28.0355
Iteration 57: Best length = 28.0355
Iteration 58: Best length = 28.0355
Iteration 59: Best length = 28.0355
Iteration 60: Best length = 28.0355
Iteration 61: Best length = 28.0355
Iteration 62: Best length = 28.0355
Iteration 63: Best length = 28.0355
Iteration 64: Best length = 28.0355
Iteration 65: Best length = 28.0355
Iteration 66: Best length = 28.0355
Iteration 67: Best length = 28.0355
Iteration 68: Best length = 28.0355
Iteration 69: Best length = 28.0355
Iteration 70: Best length = 28.0355
Iteration 71: Best length = 28.0355
Iteration 72: Best length = 28.0355
Iteration 73: Best length = 28.0355
Iteration 74: Best length = 28.0355
Iteration 75: Best length = 28.0355
Iteration 76: Best length = 28.0355
Iteration 77: Best length = 28.0355
Iteration 78: Best length = 28.0355
Iteration 79: Best length = 28.0355
Iteration 80: Best length = 28.0355
Iteration 81: Best length = 28.0355
Iteration 82: Best length = 28.0355
Iteration 83: Best length = 28.0355
Iteration 84: Best length = 28.0355
Iteration 85: Best length = 28.0355
Iteration 86: Best length = 28.0355
Iteration 87: Best length = 28.0355
Iteration 88: Best length = 28.0355
Iteration 89: Best length = 28.0355
Iteration 90: Best length = 28.0355
Iteration 91: Best length = 28.0355
Iteration 92: Best length = 28.0355
Iteration 93: Best length = 28.0355
Iteration 94: Best length = 28.0355

Iteration 95: Best length = 28.0355
Iteration 96: Best length = 28.0355
Iteration 97: Best length = 28.0355
Iteration 98: Best length = 28.0355
Iteration 99: Best length = 28.0355
Iteration 100: Best length = 28.0355

Best path found:

[3, np.int64(5), np.int64(6), np.int64(1), np.int64(0), np.int64(2), np.int64(4)]

Best path length: 28.0355

Program 5

Cuckoo Search

Algorithm:

Cuckoo Search

Pseudocode

- 1.) Set initial number of nests n , probability $P \in (0, 1)$ and maximum iterations $MaxIter$.
- 2.) Initialize iteration counter $t = 0$
- 3.) ~~for~~ $i = 1$ ~~do~~ n ~~do~~
 - 4.) Generate an initial solution x_i as a binary vector representing feature subset.
 - 5.) Evaluate fitness $f(x_i)$ based on model accuracy with selected features.
- 6.) ~~End for~~
- 7.) ~~while~~ $t < MaxIter$ ~~do~~
 - 8.) Generate a new solution x_i by performing levy flight
 - 9.) Evaluate fitness $f(x_i)$
 - 10.) Randomly select a nest x_j among n nests
 - 11.) If $f(x_i) < f(x_j)$ then
 - 12.) replace x_j with x_i
 - 13.) ~~End if~~
 - 14.) Abandon a fraction a of worst nests and generate new nests randomly
 - 15.) Keep the best seen found so far
 - 16.) Rank all nests by fitness and update worst best.
 - 17.) increment iteration $t = t + 1$
 - 18.) ~~End while~~
 - 19.) Off the best feature subset x_{best} .

O/P

Iter 1, best fitness : 0.4039, selected 14
Iter 2, best : 0.44 selected 14

Iter 50, Best : ~~0.5558~~, selected : 18

Best feature subset found : [0 1 1 1 1 1 1 1 0

1 1 1 1 1 1 1 1]
1

Code:

```
import numpy as np
import math
```

Example objective function: Sphere function (min at 0)

```
def objective_function(x):
    return np.sum(x**2)
```

Correct Lévy Flight using Mantegna's Algorithm

```
def levy_flight(lambda_, dim):
    sigma = (math.gamma(1 + lambda_) * math.sin(math.pi * lambda_ / 2) /
              (math.gamma((1 + lambda_) / 2) * lambda_ * 2 ** ((lambda_ - 1) / 2))) ** (1 / lambda_)

    u = np.random.randn(dim) * sigma
    v = np.random.randn(dim)

    step = u / (np.abs(v) ** (1 / lambda_))
    return step
```

```
def Cuckoo_Search(n=20, dim=2, max_iter=100, pa=0.25, alpha=0.01, lb=-5, ub=5):
```

Initialize nests

```
nests = np.random.uniform(lb, ub, (n, dim))
fitness = np.array([objective_function(nest) for nest in nests])
```

Identify best nest

```
best_index = np.argmin(fitness)
best_nest = nests[best_index].copy()
best_fitness = fitness[best_index]
```

```
for t in range(max_iter):
```

```
    for i in range(n):
```

Lévy flight step

```
    step = levy_flight(1.5, dim)
    new_solution = nests[i] + alpha * step
    new_solution = np.clip(new_solution, lb, ub)
```

```
    new_fit = objective_function(new_solution)
```

Choose random nest

```
    j = np.random.randint(n)
```

Replace if improves

```
    if new_fit < fitness[j]:
        nests[j] = new_solution
        fitness[j] = new_fit
```

Abandon Pa fraction of nests

```
    for i in range(n):
```

```
        if np.random.rand() < pa:
            nests[i] = np.random.uniform(lb, ub, dim)
            fitness[i] = objective_function(nests[i])
```



```

    # Update global best
    best_index = np.argmin(fitness)
    if fitness[best_index] < best_fitness:
        best_fitness = fitness[best_index]
        best_nest = nests[best_index].copy()

    return best_nest, best_fitness

# Run the Algorithm
best_solution, best_value = Cuckoo_Search()

print("✅ Best Solution Found:", best_solution)
print("🏆 Best Fitness:", best_value)

✅ Best Solution Found: [-0.00050611  0.00096219]
🏆 Best Fitness: 1.1819556829903518e-06

```

Program 6

Grey Wolf

Algorithm:

Grey Wolf

Initialize:

$n \leftarrow$ number of wolves

number $\leftarrow$ number of iterations

$a \leftarrow 2$

For each wolf $i = 1$ to n :

Initialize position x_i randomly

Evaluate fitness $f(x_i)$

Identify α, β, δ

Repeat for $t = 1$ to number;

For each wolf $i = 1$ to n :

For each dimension j :

Generate random numbers r_1, r_2 in $[0, 1]$

Compute $A = 2 * a * r_1 - a$

Compute $C = 2 * r_2$

Compute distances from α, β, δ :

$D_\alpha = |(1 * x_\alpha - x_i)|$

$D_\beta = |(1 * x_\beta - x_i)|$

$D_\delta = |(1 * x_\delta - x_i)|$

Compute three candidate positions:

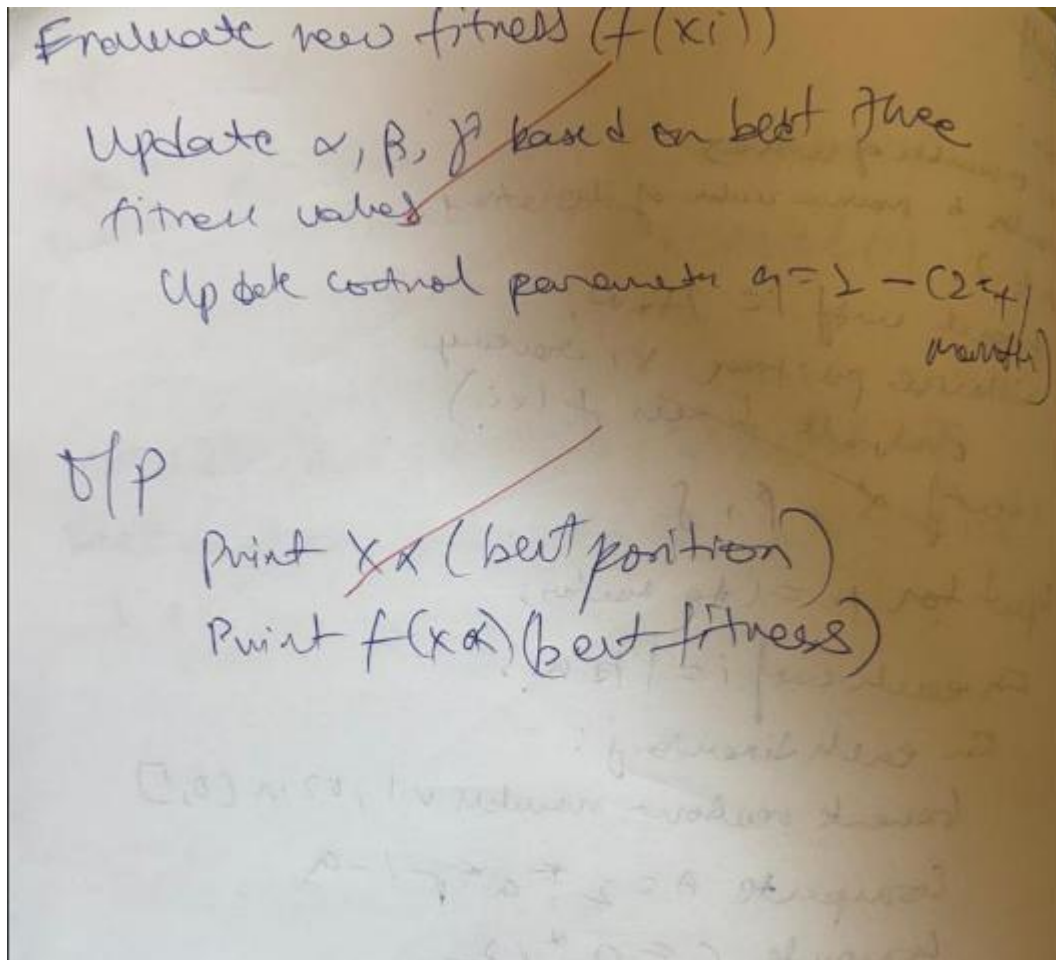
$x_1 = x_\alpha - A_1 * D_\alpha$

$x_2 = x_\beta - A_2 * D_\beta$

$x_3 = x_\delta - A_3 * D_\delta$

Update wolf position:

$x_i = (x_1 + x_2 + x_3) / 3$



Code:

```
import numpy as np
```

```
# Example objective function: Sphere function (min at 0)
```

```
# Replace this with your actual function f(X)
```

```
def objective_function(X):
```

```
    return np.sum(X**2)
```

```
def Grey_Wolf_Optimizer(N=30, dim=2, max_iter=100, lb=-10, ub=10):
```

```
    # Initialize wolf positions
```

```
    Wolves = np.random.uniform(lb, ub, (N, dim))
```

```
    Fitness = np.array([objective_function(w) for w in Wolves])
```

```
    # Identify  $\alpha, \beta, \delta$  (best three wolves)
```

```
    sorted_idx = np.argsort(Fitness)
```

```
    Alpha = Wolves[sorted_idx[0]].copy()
```

```
    Beta = Wolves[sorted_idx[1]].copy()
```

```
    Delta = Wolves[sorted_idx[2]].copy()
```

```
    Alpha_score = Fitness[sorted_idx[0]]
```

```

Beta_score = Fitness[sorted_idx[1]]
Delta_score = Fitness[sorted_idx[2]]

# Main optimization loop
for t in range(max_iter):
    a = 2 - (2 * t / max_iter) # linearly decreases from 2 → 0

    for i in range(N):
        for j in range(dim):

            r1, r2 = np.random.rand(), np.random.rand()
            A1 = 2 * a * r1 - a
            C1 = 2 * r2

            D_alpha = abs(C1 * Alpha[j] - Wolves[i, j])
            X1 = Alpha[j] - A1 * D_alpha

            r1, r2 = np.random.rand(), np.random.rand()
            A2 = 2 * a * r1 - a
            C2 = 2 * r2

            D_beta = abs(C2 * Beta[j] - Wolves[i, j])
            X2 = Beta[j] - A2 * D_beta

            r1, r2 = np.random.rand(), np.random.rand()
            A3 = 2 * a * r1 - a
            C3 = 2 * r2

            D_delta = abs(C3 * Delta[j] - Wolves[i, j])
            X3 = Delta[j] - A3 * D_delta

            # Update wolf's new position
            Wolves[i, j] = (X1 + X2 + X3) / 3

# Ensure bounds
Wolves = np.clip(Wolves, lb, ub)

# Re-evaluate fitness
Fitness = np.array([objective_function(w) for w in Wolves])

# Update  $\alpha$ ,  $\beta$ ,  $\delta$ 
sorted_idx = np.argsort(Fitness)

if Fitness[sorted_idx[0]] < Alpha_score:
    Alpha = Wolves[sorted_idx[0]].copy()
    Alpha_score = Fitness[sorted_idx[0]]

if Fitness[sorted_idx[1]] < Beta_score:
    Beta = Wolves[sorted_idx[1]].copy()

```

```

    Beta_score = Fitness[sorted_idx[1]]

    if Fitness[sorted_idx[2]] < Delta_score:
        Delta = Wolves[sorted_idx[2]].copy()
        Delta_score = Fitness[sorted_idx[2]]

    return Alpha, Alpha_score

# Run the optimizer
best_pos, best_fit = Grey_Wolf_Optimizer()

print("Best Position ( $X\alpha$ ):", best_pos)
print("Best Fitness  $f(X\alpha)$ :", best_fit)

Best Position ( $X\alpha$ ): [-6.48978187e-29 -5.86094241e-29]
Best Fitness  $f(X\alpha)$ : 7.64679146961405e-57

```

Program 7 Parallel Cellular

Algorithm:

Parallel Cellular

1. Initialize:

Define the optimization problem: minimize $f(n)$

Set parameters:

num - cells $\leftarrow$ number of cells

grid - size $\leftarrow$ (rows, columns)

neighbourhood $\leftarrow$ ~~structure~~

max_iter $\leftarrow$ number of iterations

For each cell $i=1$ to num cells:

Initialize position x_i randomly

Evaluate fitness $f(n_i)$

Identify ~~p~~ best-cell $\leftarrow$ cell with best fitness

2. Repeat for $t=1$ to max_iter:

For each cell i (in parallel):

Step 1: neighbourhood interaction

Identify ~~neighbouring~~ cells N_i

Step 2: Apply update rule

n_i - new $\leftarrow$ update (x_i, N_i)

n_i - new $= n_i + r^k (n_{\text{best-neighbour}} - n_i)$

Evaluate fitness $f(ni_new)$

Step 3: Selection

$r + f(ni_new) \geq f(ni)$

$ni \leftarrow ni_new$

Step 4: Update global best

if any $f(ni) < f(best_cell)$:

$best_cell \leftarrow ni$

3.) Output

Print $best_cell$ (best solution)

Print $f(best_cell)$ (best fitness)

End

O/p

Best Solution Found : [0, 0.3550541...]
Best fitness : 0.0015374121 3005402448

Code:

```
import numpy as np

def objective_function(x):
    """Example objective function: Sphere function (min at 0).
    Replace this with your actual f(x)."""
    return np.sum(x**2)

def get_neighbors(index, grid_size):
    """Returns Moore neighborhood indices of a cell index."""
    rows, cols = grid_size
    r, c = index

    neighbors = []
    for dr in [-1, 0, 1]:
        for dc in [-1, 0, 1]:
            if dr == 0 and dc == 0:
                continue
            nr, nc = r + dr, c + dc
            if 0 <= nr < rows and 0 <= nc < cols:
                neighbors.append((nr, nc))
    return neighbors

def Parallel_Cellular_Algorithm(grid_size=(5,5), dim=2, max_iter=100, r=0.5):
    rows, cols = grid_size
    num_cells = rows * cols

    # Initialize population grid
    grid = np.random.uniform(-5, 5, (rows, cols, dim))
    fitness = np.zeros((rows, cols))

    # Evaluate initial fitness
    for i in range(rows):
        for j in range(cols):
            fitness[i, j] = objective_function(grid[i, j])

    # Identify global best
    best_position = grid[fitness.argmin() // cols, fitness.argmin() % cols].copy()
    best_fitness = np.min(fitness)

    # Main Loop
    for _ in range(max_iter):

        new_grid = grid.copy()

        # Parallel cell update (simulated)
        for i in range(rows):
            for j in range(cols):
```

```

neighbors = get_neighbors((i, j), grid_size)

# Find best neighbor
best_neighbor = min(neighbors, key=lambda idx: fitness[idx[0], idx[1]])
br, bc = best_neighbor

# Update position
new_position = grid[i, j] + r * (grid[br, bc] - grid[i, j])
new_fit = objective_function(new_position)

# Selection Step
if new_fit < fitness[i, j]:
    new_grid[i, j] = new_position
    fitness[i, j] = new_fit

grid = new_grid # Update entire grid at once (parallel effect)

# Update Global Best
current_best = fitness.min()
if current_best < best_fitness:
    best_fitness = current_best
    best_pos_idx = np.unravel_index(fitness.argmin(), fitness.shape)
    best_position = grid[best_pos_idx].copy()

return best_position, best_fitness

# Run Algorithm
best_solution, best_value = Parallel_Cellular_Algorithm()

print("Best Solution Found:", best_solution)
print("Best Fitness:", best_value)

```

```

Best Solution Found: [ 0.03550541 -0.01663672]
Best Fitness: 0.0015374143005402448

```